Kubernetes Admission Control — Straight-to-the-Point Lesson

Audience: experienced DevOps / DevSecOps / Platform Security engineer refreshing Kubernetes policy controls.
Scope: Admission control, Pod Security Admission, Kyverno, ValidatingAdmissionPolicy/CEL, OPA/Gatekeeper, debugging, production rollout, and interview answers.

---

1. Admission Control Mental Model

Kubernetes admission control sits after authentication and authorization, but before an object is persisted in the API server.

```
kubectl apply
  → authentication
  → authorization / RBAC
  → admission control
  → object stored in etcd
  → controllers act on object
```

Admission control answers questions like:

- Is this Pod allowed to run as privileged?
- Is this image from an approved registry?
- Does this workload have ownership labels?
- Is this namespace enforcing the restricted Pod Security Standard?
- Should this request be denied, warned, audited, or mutated?

Admission control is not the same as runtime detection. It prevents or changes resources at API request time.

Common admission layers

Layer	Purpose
Built-in admission controllers	Native Kubernetes controls such as Pod Security Admission
Dynamic admission webhooks	External controllers such as Kyverno or Gatekeeper

ValidatingAdmissionPolicy	Native CEL-based validation inside the API server
Mutating admission	Changes objects before persistence, for example defaulting fields
Validating admission	Allows or denies objects based on policy

---

2. Key Concepts Cheat Sheet

Pod Security Standards, PSS

Pod Security Standards are Kubernetes-defined pod security profiles:

```
privileged → baseline → restricted
```

- **Privileged:** mostly unrestricted.
- **Baseline:** prevents known privilege escalation patterns while staying broadly compatible.
- **Restricted:** hardened baseline for least-privilege workloads.

PSS is the standard. It does not enforce anything by itself.

Pod Security Admission, PSA

Pod Security Admission is the built-in Kubernetes admission controller that enforces PSS through namespace labels.

```
kubectl label ns app-ns \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=v1.30 \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/warn-version=v1.30 \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/audit-version=v1.30 \
  --overwrite
```

Modes:

Mode	Behavior
`enforce`	Rejects violating Pods

`warn`	Allows request but returns client warning
`audit`	Allows request but records violation in audit logs

PSA is excellent as a baseline. It is intentionally not customizable.

Kyverno

Kyverno is a Kubernetes-native policy engine. It runs as admission webhooks/controllers and uses YAML policies.

It can:

- validate resources
- mutate resources
- generate resources
- verify images/signatures
- produce policy reports
- support richer exception workflows

Kyverno is practical when platform teams want readable YAML policies and good developer feedback.

ValidatingAdmissionPolicy, VAP

ValidatingAdmissionPolicy is native Kubernetes admission validation using CEL.

Two objects matter:

```
ValidatingAdmissionPolicy         = policy logic
ValidatingAdmissionPolicyBinding  = scope and enforcement action
```

Binding actions:

```
validationActions: [Deny]
```

or:

```
validationActions: [Warn, Audit]
```

VAP is good for simple, stable validation without an external webhook controller.

CEL

CEL is the expression language used by VAP.

Example:

```
!has(object.spec.volumes) || object.spec.volumes.all(v, !has(v.hostPath))
```

CEL is good for field checks. It gets harder to maintain when logic becomes deeply nested or exception-heavy.

OPA / Gatekeeper / Rego

- **OPA** is the policy engine.
- **Rego** is the policy language.
- **Gatekeeper** is the Kubernetes admission-controller integration.

Gatekeeper is strong when an organization already uses OPA/Rego across multiple systems, not only Kubernetes.

---

3. Tool Comparison

Dimension	PSA	Kyverno	CEL / VAP	Gatekeeper
Main use	Built-in pod security baseline	Kubernetes-native policy lifecycle	Native simple validation	OPA/Rego policy enforcement
Runs where	API server	Webhook/controller	API server	Webhook/controller
Custom logic	No	Yes	Yes, limited by CEL readability	Yes, highly expressive

Mutation	No	Yes	No	Limited / possible depending setup
Reports	Audit logs only	PolicyReports	Audit logs	Audit/status depending setup
Exception model	Namespace-level/coarse	Stronger, policy-specific	Scope/params, not rich lifecycle	Strong with Rego/constraints
Operational burden	Very low	Medium	Low	Medium-high
Best for	Baseline hardening	Platform policies	Simple native guardrails	Enterprise OPA strategy

Decision tree

```
Need only baseline/restricted Pod Security?
  → PSA

Need simple field validation without a webhook?
  → ValidatingAdmissionPolicy + CEL

Need Kubernetes-native policy lifecycle, reports, mutation, exceptions, image verificat
  → Kyverno

Need highly expressive policy or one policy language across Kubernetes, Terraform, CI,
  → OPA/Gatekeeper
```

Interview line:

> I would not treat these tools as mutually exclusive. PSA gives the baseline. CEL/VAP is useful for simple native validation. Kyverno is practical for Kubernetes-native policy lifecycle. Gatekeeper is strong when the organization has broader OPA/Rego investment.

---

4. PSA Cheat Sheet

Enable restricted in warn/audit first

```
kubectl label ns policy-lab \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/warn-version=latest \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/audit-version=latest \
  --overwrite
```

Move to enforce

```
kubectl label ns policy-lab \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=latest \
  --overwrite
```

Inspect labels

```
kubectl get ns policy-lab --show-labels
kubectl describe ns policy-lab
```

Remove enforcement

```
kubectl label ns policy-lab \
  pod-security.kubernetes.io/enforce- \
  pod-security.kubernetes.io/enforce-version- \
  --overwrite
```

Common restricted requirements

Restricted usually expects controls like:

```yaml
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
```

```
    containers:
      - name: app
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop:
              - ALL
```

Restricted also blocks patterns like:

- privileged containers
- hostPath volumes
- host namespaces
- unrestricted Linux capabilities
- missing or unsafe seccomp settings
- privilege escalation

PSA debugging pattern

If a direct Pod is blocked:

```
kubectl apply -f pod.yaml
```

Expected error shape:

```
Error from server (Forbidden): pods "x" is forbidden:
violates PodSecurity "restricted:latest": ...
```

If a Deployment is accepted but no Pods appear:

```
kubectl describe deploy -n <ns> <deployment>
kubectl get rs -n <ns>
kubectl describe rs -n <ns> <replicaset>
kubectl get events -n <ns> --sort-by=.lastTimestamp
kubectl get ns <ns> --show-labels
```

Expected ReplicaSet event:

```
Warning FailedCreate replicaset-controller
Error creating: pods "x" is forbidden: violates PodSecurity "restricted:latest": ...
```

Key point:

```
Deployment accepted does not mean Pod creation succeeded.
PSA enforce applies when Pods are created.
```

---

5. Kyverno Cheat Sheet

Install with Helm

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update

helm install kyverno kyverno/kyverno \
  --namespace kyverno \
  --create-namespace
```

Verify

```
kubectl -n kyverno get pods
kubectl -n kyverno get deploy
kubectl -n kyverno rollout status deploy/kyverno-admission-controller
kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations | grep kyvern
```

Basic ClusterPolicy shape

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-standard-labels
spec:
```

```
  background: false
  rules:
    - name: require-owner-label
      match:
        any:
          - resources:
              kinds:
                - Pod
              namespaces:
                - kyverno-lab
      validate:
        failureAction: Enforce
        message: "Pods must include owner label."
        pattern:
          metadata:
            labels:
              owner: "?*"
```

Audit vs enforce

```
failureAction: Audit
```

Allows but reports.

```
failureAction: Enforce
```

Blocks admission.

Allowed registries pattern

```
validate:
  failureAction: Enforce
  message: "Images must come from approved registries."
  foreach:
    - list: "request.object.spec.containers"
      deny:
        conditions:
          all:
            - key: "{{ regex_match('^(registry\\.k8s\\.io/|ghcr\\.io/my-org/)', element
              operator: Equals
              value: false
```

```
    - list: "request.object.spec.initContainers || `[]`"
      deny:
        conditions:
          all:
            - key: "{{ regex_match('^(registry\\.k8s\\.io/|ghcr\\.io/my-org/)', element
              operator: Equals
              value: false
    - list: "request.object.spec.ephemeralContainers || `[]`"
      deny:
        conditions:
          all:
            - key: "{{ regex_match('^(registry\\.k8s\\.io/|ghcr\\.io/my-org/)', element
              operator: Equals
              value: false
```

Debug Kyverno

```
kubectl get cpol,pol -A
kubectl describe cpol <policy>
kubectl -n kyverno get pods
kubectl -n kyverno logs deploy/kyverno-admission-controller --tail=100
kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations | grep kyvern
kubectl get policyreport -A
```

Kyverno denial usually looks like:

```
admission webhook "validate.kyverno..." denied the request
resource ... was blocked due to the following policies
```

Production Kyverno notes

Care about:

- webhook availability
- failure policy
- namespace exclusions
- HA replicas
- policy testing in CI
- PolicyReports

- PolicyException governance
- emergency rollback

Rollback options:

```
kubectl patch cpol <policy> \
  --type='json' \
  -p='[{"op":"replace","path":"/spec/rules/0/validate/failureAction","value":"Audit"}]'
```

or:

```
kubectl delete cpol <policy>
```

---

6. CEL / ValidatingAdmissionPolicy Cheat Sheet

Check support

```
kubectl api-resources | grep -i validatingadmission
kubectl version
```

You want:

```
validatingadmissionpolicies
validatingadmissionpolicybindings
```

Required labels example

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: required-labels.example.com
spec:
  failurePolicy: Fail
```

```
  matchConstraints:
    resourceRules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["pods"]
  validations:
    - expression: >
        has(object.metadata.labels)
        && 'app.kubernetes.io/name' in object.metadata.labels
        && 'owner' in object.metadata.labels
        && 'env' in object.metadata.labels
      message: "Pods must include app.kubernetes.io/name, owner, and env labels."
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: required-labels-binding.example.com
spec:
  policyName: required-labels.example.com
  validationActions: [Deny]
  matchResources:
    namespaceSelector:
      matchLabels:
        security.example.com/policy: enabled
```

Disallow hostPath

```
validations:
  - expression: >
      !has(object.spec.volumes)
      || object.spec.volumes.all(v, !has(v.hostPath))
    message: "hostPath volumes are not allowed."
```

This is a clean CEL use case.

Allowed registries

```
validations:
  - expression: >
      object.spec.containers.all(c,
        c.image.startsWith('registry.k8s.io/')
        || c.image.startsWith('ghcr.io/my-org/')
      )
```

```
      &&
      (!has(object.spec.initContainers)
        || object.spec.initContainers.all(c,
          c.image.startsWith('registry.k8s.io/')
          || c.image.startsWith('ghcr.io/my-org/')
        )
      )
      &&
      (!has(object.spec.ephemeralContainers)
        || object.spec.ephemeralContainers.all(c,
          c.image.startsWith('registry.k8s.io/')
          || c.image.startsWith('ghcr.io/my-org/')
        )
      )
    message: "All container images must come from approved registries."
```

Switch from deny to warn/audit

Change the binding, not the policy:

```
validationActions: [Warn, Audit]
```

Blocking:

```
validationActions: [Deny]
```

Do not combine `Deny` and `Warn` for the same binding.

CEL rules of thumb

Use CEL when:

- the rule is simple
- the rule is stable
- the object field path is clear
- validation is enough
- no rich exception lifecycle is needed
- avoiding webhook dependency matters

Avoid CEL or reconsider when:

- expressions become deeply nested
- you need mutation
- you need reports
- you need image signature verification
- you need complex exceptions
- you need reusable policy libraries

Interview line:

> I like CEL/VAP for simple, native guardrails. If the expression becomes a mini-program, I would consider Kyverno or Gatekeeper for readability and lifecycle management.

---

7. Debugging Admission Failures

First classify the failure

```
Object rejected immediately?
  → admission denied the submitted object

Deployment accepted but no Pods?
  → ReplicaSet likely failed to create Pods

Pod ImagePullBackOff?
  → node/runtime/registry problem, not admission policy

Policy not firing?
  → scope, binding, match/exclude, controller health, or mode issue
```

Identify the denying layer

Error text	Likely layer
`violates PodSecurity "restricted"`	PSA
`admission webhook "validate.kyverno..." denied`	Kyverno
`admission webhook "validation.gatekeeper..." denied`	Gatekeeper
`ValidatingAdmissionPolicy ... denied`	CEL / VAP
`forbidden: User ... cannot ...`	RBAC, not admission policy

`ImagePullBackOff`	Node/runtime/image pull issue

Debug command set

```
kubectl get ns --show-labels
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe deploy -n <ns> <name>
kubectl get rs -n <ns>
kubectl describe rs -n <ns> <name>
kubectl describe pod -n <ns> <name>
kubectl get cpol,pol -A
kubectl get validatingadmissionpolicy,validatingadmissionpolicybinding
kubectl -n kyverno logs deploy/kyverno-admission-controller --tail=100
```

Common debugging scenarios

Scenario: Deployment created but no Pods

Likely cause: Pod template violates PSA/Kyverno/CEL.

Inspect:

```
kubectl describe deploy -n <ns> <deploy>
kubectl get rs -n <ns>
kubectl describe rs -n <ns> <rs>
kubectl get events -n <ns> --sort-by=.lastTimestamp
```

Scenario: Kyverno policy does not block

Check:

```
kubectl get cpol <policy> -o yaml
kubectl describe cpol <policy>
kubectl -n kyverno get pods
kubectl get validatingwebhookconfigurations | grep kyverno
```

Likely causes:

- policy is in `Audit`, not `Enforce`

- namespace not matched
- kind not matched
- namespace excluded
- webhook/controller not ready
- object existed before policy and background scanning is disabled

Scenario: CEL expression misses init containers

Bad pattern:

```
object.spec.containers.all(...)
```

Better pattern:

```
containers + initContainers + ephemeralContainers
```

Always think about all execution paths.

Scenario: Missing field breaks CEL logic

Use `has()`:

```
!has(c.securityContext)
|| !has(c.securityContext.privileged)
|| c.securityContext.privileged == false
```

---

8. Production Rollout Plan

Goal

Raise the default Kubernetes workload security baseline without surprising teams, breaking production, or creating a policy system nobody trusts.

Recommended sequence

```
1. Inventory workloads and namespaces
2. Identify owners and criticality
3. Enable warn/audit first
4. Report violations to teams
5. Fix common issues and provide examples
6. Add CI/pre-commit validation
7. Enforce in new/dev namespaces
8. Enforce in staging and low-risk production
9. Expand namespace by namespace
10. Maintain exception and rollback process
```

Inventory

Collect:

- namespaces and owners
- privileged containers
- hostPath usage
- images and registries
- missing `runAsNonRoot`
- missing seccomp
- missing `allowPrivilegeEscalation: false`
- capabilities not dropped
- system/agent workloads
- deployment mechanism

Useful commands:

```
kubectl get ns --show-labels
kubectl get deploy,sts,ds,job,cronjob -A
kubectl get pods -A -o wide
kubectl get pods -A -o json | jq -r '.items[].spec.containers[]?.image' | sort -u
```

Rollout by namespace type

Namespace type	Rollout approach
New app namespaces	Enforce early

Dev namespaces	Audit/warn, then enforce quickly
Staging	Enforce after fixes are proven
Production apps	Gradual, owner-approved rollout
System namespaces	Separate review
Security/observability agents	Specific exceptions may be valid
Legacy apps	Longer remediation path

Exceptions

A good exception includes:

- workload name
- namespace
- owner
- policy and rule
- business reason
- risk acceptance
- compensating controls
- expiration date
- reviewer
- ticket/Git reference

Bad exception:

```
Exclude this entire namespace from everything forever.
```

Better exception:

```
Allow hostPath /var/log only for fluent-bit DaemonSet in logging namespace until 2026-0
```

CI before admission

Developers should see failures before admission blocks deployment.

Good feedback order:

```
pre-commit → PR/CI → rendered manifest validation → server-side dry-run → admission enf
```

Admission control should be the final boundary, not the first feedback mechanism.

Metrics

Track:

- number of violating workloads
- violations by team/namespace
- privileged workloads remaining
- hostPath workloads remaining
- unapproved registries
- exception count and age
- admission denial count
- Kyverno webhook latency/errors
- policy-related deployment failures
- percentage of namespaces at baseline/restricted

Emergency rollback

PSA:

```
kubectl label ns <ns> pod-security.kubernetes.io/enforce- --overwrite
```

Kyverno:

```
kubectl patch cpol <policy> \
  --type='json' \
  -p='[{"op":"replace","path":"/spec/rules/0/validate/failureAction","value":"Audit"}]'
```

VAP:

```
kubectl patch validatingadmissionpolicybinding <binding> \
  --type='json' \
  -p='[{"op":"replace","path":"/spec/validationActions","value":["Warn","Audit"]}]'
```

Senior framing:

> Admission control rollout is production change management. The syntax matters, but the real work is ownership, communication, exceptions, CI feedback, observability, and rollback.

---

9. Interview Questions and Concise Answers

1. What is Kubernetes admission control?

Admission control is the stage after authentication and authorization but before persistence where Kubernetes can validate, reject, or mutate API requests.

2. What is the difference between PSS and PSA?

PSS defines the `privileged`, `baseline`, and `restricted` pod security profiles. PSA is the built-in admission controller that enforces those profiles through namespace labels.

3. When would you use PSA?

I would use PSA as a built-in pod security baseline, especially for enforcing `baseline` or `restricted` profiles with minimal operational overhead.

4. Why is PSA insufficient for approved registries?

PSA only enforces predefined Pod Security Standards. Approved registries are organization-specific supply-chain policy, so I would use Kyverno, CEL/VAP, or Gatekeeper.

5. When would you choose Kyverno over CEL?

I would choose Kyverno when I need Kubernetes-native policy lifecycle features like reports, mutation, image verification, richer exceptions, or clearer developer workflows.

6. When is CEL/VAP a good fit?

CEL/VAP is a good fit for simple, stable validation rules that can be expressed directly against the Kubernetes object and do not require an external webhook.

7. When would Gatekeeper still be a strong choice?

Gatekeeper is strong when the organization already uses OPA/Rego or needs highly expressive reusable policy across Kubernetes and other systems.

8. How would you roll out restricted pod security without breaking production?

I would inventory workloads, enable warn/audit first, communicate with teams, add CI validation, fix common issues, handle narrow exceptions, and enforce namespace by namespace with rollback ready.

9. What is the risk of enforcing restricted everywhere on day one?

It can break existing workloads, especially legacy apps, DaemonSets, agents, or system components, and it can damage trust if teams are surprised by blocked deployments.

10. How do you debug a Deployment that was created but has no Pods?

I would inspect the Deployment, ReplicaSet, namespace labels, and events. The ReplicaSet often shows `FailedCreate` if Pod admission was denied.

11. How do you distinguish PSA from Kyverno denials?

PSA usually says `violates PodSecurity "restricted"` or `baseline`. Kyverno usually says `admission webhook "validate.kyverno..." denied` and includes policy/rule names.

12. Why must policies check initContainers and ephemeralContainers?

Because they are also execution paths. If a policy checks only `spec.containers`, images or security settings can bypass policy through init or ephemeral containers.

13. How do you switch VAP from blocking to warn/audit?

Change `validationActions` in the `ValidatingAdmissionPolicyBinding`, not the CEL expression. Use `[Deny]` to block or `[Warn, Audit]` to observe.

14. How do you handle exceptions safely?

Use narrow, reviewed, time-bound exceptions scoped to a specific workload, namespace, policy, and rule, with owner, justification, compensating controls, and expiry.

15. How do you balance security with developer velocity?

Give feedback early in CI, provide clear error messages and examples, start in audit mode, fix common patterns through templates, and reserve admission denial for clear, communicated controls.

16. Why not rely only on admission control?

Admission prevents new bad resources, but you still need CI validation, runtime detection, image scanning, audit logs, drift detection, exception review, and incident response.

17. What is your preferred layered model?

PSA for baseline pod security, CEL/VAP for simple native validation, Kyverno for Kubernetes-native policy lifecycle, and Gatekeeper when OPA/Rego fits the broader organization strategy.

18. How would you explain this to a non-security platform team?

These controls create safe defaults for workloads so teams can ship without accidentally using risky settings like privileged containers, hostPath volumes, or unapproved images.

19. How would you migrate from custom OPA policies to newer native controls?

I would inventory existing OPA policies, map simple pod security rules to PSA, map simple validations to VAP/CEL, keep complex or cross-platform rules in Gatekeeper, and migrate gradually with tests and audit mode.

20. What is a good senior summary of admission control strategy?

Use the simplest reliable control that solves the problem: built-in controls for baselines, native CEL for simple validation, and policy engines when lifecycle, exceptions, reporting, or expressiveness matter.

---

10. High-Signal Phrases for Interviews

Use these naturally:

- “PSA is a baseline, not a complete policy program.”
- “Admission control is the final enforcement boundary; CI should provide earlier feedback.”
- “I would start in audit/warn mode before enforcing.”
- “I would avoid broad namespace exceptions and prefer scoped, time-bound exceptions.”
- “The first debugging step is identifying which admission layer denied the request.”
- “If a Deployment exists but no Pods start, I check ReplicaSet events.”
- “For container policies, I always check containers, initContainers, and ephemeralContainers.”
- “CEL is great until the expression becomes a mini-program.”
- “Kyverno is practical when policy lifecycle matters.”
- “Gatekeeper makes sense when OPA/Rego is already part of the organization’s governance model.”

11. Final Memory Model

```
PSA:
  Built-in baseline pod hardening.

Kyverno:
  Kubernetes-native policy lifecycle, reports, mutation, exceptions, image verification

CEL / VAP:
  Native simple validation without webhook dependency.

Gatekeeper:
  OPA/Rego power, reusable policy, broader governance strategy.

Production rollout:
  Inventory → audit/warn → fix → CI feedback → scoped enforce → exceptions → metrics →

Debugging:
  Identify the denying layer first.
```

Sources / Further Reading

- Kubernetes documentation: Pod Security Admission
- Kubernetes documentation: Enforce Pod Security Standards with Namespace Labels
- Kubernetes documentation: Validating Admission Policy
- Kubernetes blog: Validating Admission Policy GA in Kubernetes 1.30
- Kyverno documentation: Validate rules, installation, reports, exceptions
- Kyverno project documentation: policy-as-code, validation, mutation, generation, image verification
- Gatekeeper documentation: ConstraintTemplates and Constraints